LLH Low-Level Security Track: LeakScope Problem Statement

LeakScope: overview

Build a tool that finds secret-dependent timing leaks in compiled programs. How you do that is up to you.

Encryption can be mathematically sound and still give away its key. If a program behaves differently depending on a secret, an attacker who can measure time or watch the CPU cache can recover that secret. Compilers can introduce these leaks into code that looked safe in its source, so the source alone cannot be trusted.

The challenge

Build a tool that takes a compiled x86-64 Linux program with no source code and decides whether its running behaviour depends on a secret. Each target comes with a short description of its entry point and which inputs are secret.

Your tool should:

1. **Find every kind of leak.** Catch secret-dependent branches, secret-dependent memory accesses and variable-time instructions, including speculative leaks that never show up in a normal run.
2. **Pinpoint the leak.** Report the address of each leak and what kind it is.
3. **Prove it.** For each leak, produce two secret values that make the program behave differently. For a speculative leak, also give the mispredictions that trigger it. A claim without proof does not count.
4. **Avoid false alarms.** Safe code must be reported as safe, even when it looks suspicious.
5. **Handle real compiled code.** Work across compilers and optimization levels, and follow secrets through memory, registers, CPU flags and function calls.
6. **Finish in a reasonable time.** Analysis must complete without human help.

Go further

The best entries will surprise us. One direction is automatically repairing a leak in the binary while keeping the program correct. Build what you think matters most.

Submission

Submit one GitHub repository. Everything must be inside it.

Item	What your repository must include
Code	All source code, with its commit history
Run instructions	All dependencies and setup steps, clearly listed, plus how to build it, how to run it on a target, and which Linux distributions it works on
Evidence	The tests and benchmarks that show it works
Demo	A link to a 5-minute video of the working system

Evaluation

Total score: 100.

Criterion	Score
Correctness	25
Technical depth	25
Robustness	15
Innovation	10
Performance	10
Demonstration	10
Code quality	5

Rules

Any language, public documentation and low-level library is allowed, including Capstone, Unicorn and libraries like them. The analysis itself must be your own work, not a fork or wrapper of an existing analysis tool such as ctgrind, dudect, Microwalk, Binsec, angr, Spectector, Pitchfork or Teapot.

Your tool must run offline, with no LLM, API or other network calls.

You may be required to give a short viva if the organizers request one. Failing to do so may result in your submission being rejected.